



ASCI

UML class relationships



UML class relationships

UML (Unified Modeling Language) adalah bahasa pemodelan standar visual yang digunakan untuk merancang, mendokumentasikan, dan memvisualisasikan sistem perangkat lunak berorientasi objek

Class relationships (hubungan antar kelas) adalah koneksi atau relasi logis antara satu kelas dengan kelas lainnya dalam pemrograman berorientasi objek. Relasi ini menentukan bagaimana objek-objek saling berinteraksi, berbagi data, atau saling bergantung.

Jadi UML class relationships adalah cara standar dalam UML Class Diagram untuk menggambarkan hubungan antar kelas menggunakan simbol dan garis tertentu.

Mengapa Butuh Lebih dari Satu Kelas?

Pada materi sebelumnya kita selalu bekerja dengan satu kelas dalam satu waktu. Namun di dunia nyata, sebuah sistem dibangun dari banyak kelas yang saling berinteraksi.

Sistem perbankan menggambarkan pola interaksi ini:

- Seorang **Nasabah** menggunakan **Mesin ATM** saat melakukan transaksi tunai
- Sebuah **Bank** memiliki banyak **Karyawan** yang bertugas
- Sebuah **Rekening** terdiri dari kumpulan **Catatan Transaksi** mutasi

Setiap baris di atas menggambarkan hubungan antar kelas. Di pemrograman berorientasi objek, terdapat tiga jenis hubungan utama antar kelas:

Jenis	Kata Kunci	Kekuatan	Contoh Nyata
Asosiasi	"menggunakan"	Lemah	Nasabah menggunakan Mesin ATM
Agregasi	"memiliki"	Sedang	Bank memiliki Karyawan



Komposisi	"terdiri dari"	Kuat	Rekening terdiri dari Catatan Transaksi
-----------	----------------	------	---

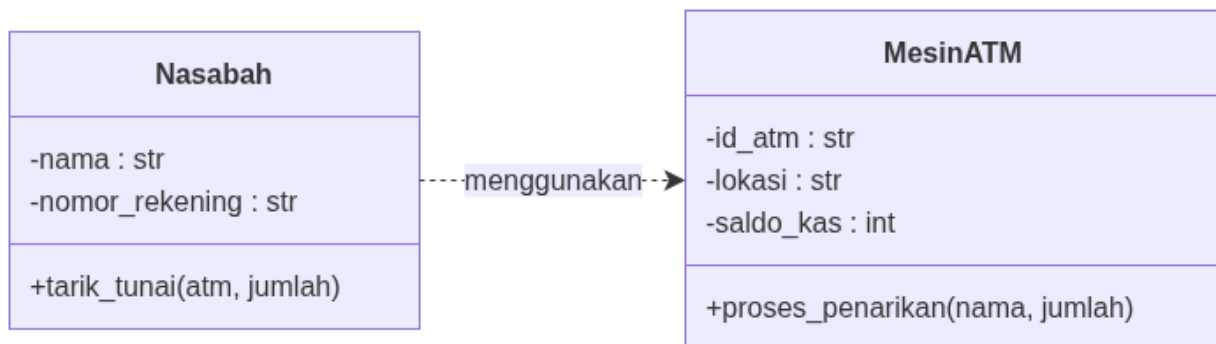
1. Asosiasi

Asosiasi adalah hubungan yang paling longgar. Objek A mengenal atau menggunakan objek B, tetapi keduanya hidup secara independen. Satu objek dapat tetap ada tanpa keberadaan objek lainnya.

Ciri-ciri Asosiasi:

- Objek B diterima sebagai parameter fungsi atau method
- Objek B dibuat sementara di dalam method lalu dilepas
- Tidak ada kepemilikan permanen karena kedua objek berdiri sendiri

Diagram UML Asosiasi:



Notasi: ..> (panah putus-putus untuk asosiasi atau dependensi search)

Kunci Asosiasi: Objek MesinATM tidak disimpan sebagai atribut tetap di dalam objek Nasabah. Objek tersebut hanya dipakai sesaat melalui parameter method saat transaksi berlangsung.

Contoh Implementasi Python:



```
class MesinATM:
    def __init__(self, id_atm, lokasi, saldo_kas):
        self.id_atm = id_atm
        self.lokasi = lokasi
        self.saldo_kas = saldo_kas

    def proses_penarikan(self, nama_nasabah, jumlah):
        if jumlah <= self.saldo_kas:
            self.saldo_kas -= jumlah
            print(f" [ATM {self.id_atm}] Penarikan Rp{jumlah:,} oleh {nama_nasabah} berhasil.")
            print(f" Sisa kas di ATM {self.lokasi}: Rp{self.saldo_kas:,}")
        else:
            print(f" [ATM {self.id_atm}] Saldo kas mesin tidak mencukupi.")

class Nasabah:
    def __init__(self, nama, nomor_rekening):
        self.nama = nama
        self.nomor_rekening = nomor_rekening
        # Tidak ada self.atm = ...
        # Nasabah tidak memiliki mesin ATM secara permanen.

    def tarik_tunai(self, atm, jumlah):
        """Asosiasi: MesinATM diterima sebagai parameter dan dipakai sementara."""
        print(f" {self.nama} memasukkan kartu ke ATM unit {atm.id_atm} ({atm.lokasi})...")
        atm.proses_penarikan(self.nama, jumlah)

# Kedua objek dibuat secara independen
atm_pusat = MesinATM("ATM-01", "Kantor Cabang Sudirman", 5000000)
budi = Nasabah("Budi Santoso", "101-220-334")
siti = Nasabah("Siti Rahma", "101-445-889")

# Asosiasi berjalan saat method dipanggil
budi.tarik_tunai(atm_pusat, 500000)
# Mesin ATM yang sama bisa digunakan oleh nasabah lain
siti.tarik_tunai(atm_pusat, 1000000)
```



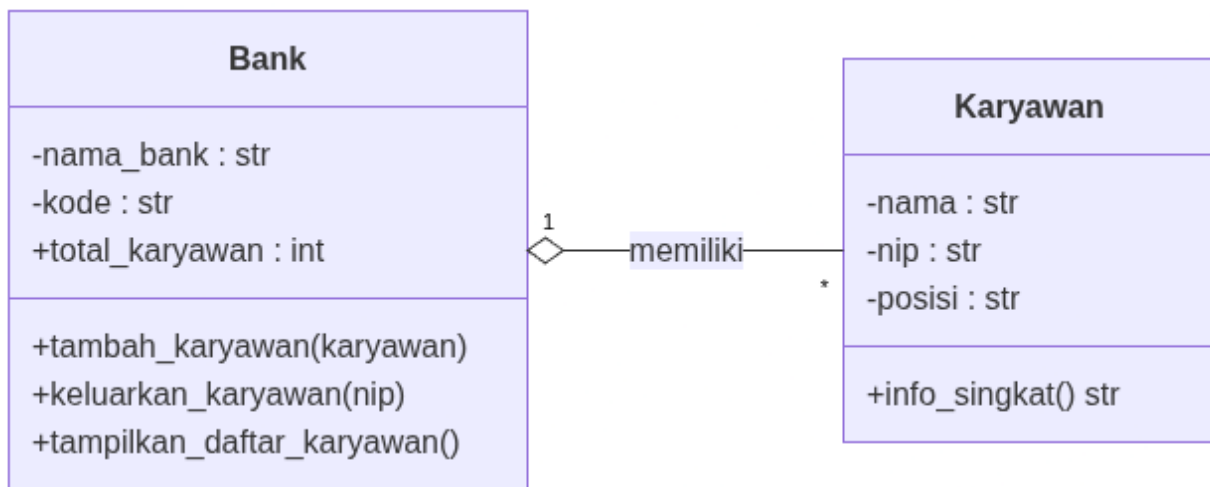
2. Agregasi

Agregasi adalah hubungan kepemilikan yang lebih kuat dari asosiasi, namun siklus hidup objek bagian tetap mandiri. Objek bagian dapat berdiri sendiri tanpa objek induknya.

Ciri-ciri Agregasi:

- Objek bagian dibuat di luar objek induk
- Objek bagian dikirim ke objek induk melalui konstruktor atau method
- Jika objek induk dihapus dari memori, objek bagian tetap ada

Diagram UML Agregasi:



Notasi: o-- (wajib kosong di sisi induk untuk agregasi)

Contoh Implementasi Python:

```
class Karyawan:
    def __init__(self, nama, nip, posisi):
        self.nama = nama
        self.nip = nip
        self.posisi = posisi
```



```
@property
def info_singkat(self):
    return f"{self.nama} ({self.posisi}) - NIP: {self.nip}"

def __str__(self):
    return f"Karyawan: {self.nama} | NIP: {self.nip} | Posisi: {self.posisi}"

class Bank:
    def __init__(self, nama_bank, kode):
        self.nama_bank = nama_bank
        self.kode = kode
        self._karyawan = [] # Agregasi: menampung referensi objek Karyawan
                             # dari luar

    def tambah_karyawan(self, karyawan):
        """Karyawan dibuat di luar dan didaftarkan ke dalam bank."""
        if isinstance(karyawan, Karyawan):
            self._karyawan.append(karyawan)
            print(f"  [+] {karyawan.nama} mulai bekerja di {self.nama_bank}")

    def keluarkan_karyawan(self, nip):
        """Melepas referensi karyawan tanpa memusnahkan objek karyawan aslinya."""
        awal = len(self._karyawan)
        self._karyawan = [k for k in self._karyawan if k.nip != nip]
        if len(self._karyawan) < awal:
            print(f"  [-] Karyawan dengan NIP {nip} telah berhenti dari {self.nama_bank}")

    @property
    def total_karyawan(self):
        return len(self._karyawan)

    def tampilkan_daftar_karyawan(self):
        print(f"\n  Daftar Pegawai {self.nama_bank} (Kode: {self.kode})")
        print(f"  Total: {self.total_karyawan} orang")
        for k in self._karyawan:
            print(f"    - {k.info_singkat}")
```



```
# Objek Karyawan dibuat secara mandiri
teller = Karyawan("Andi Wijaya", "EMP01", "Teller")
manager = Karyawan("Dewi Lestari", "EMP02", "Branch Manager")

# Objek Bank menampung karyawan
bank_mandiri = Bank("Bank Nasional Mandiri", "BNM")
bank_mandiri.tambah_karyawan(teller)
bank_mandiri.tambah_karyawan(manager)

bank_mandiri.tampilkan_daftar_karyawan()

# Bukti siklus hidup agregasi: Bank dibubarkan
del bank_mandiri

# Objek karyawan tetap utuh di memori dan dapat bekerja di tempat lain
print(f"\n Data karyawan setelah entitas bank dihapus:")
print(f" {teller}")
print(f" {manager}")
```

3. Komposisi

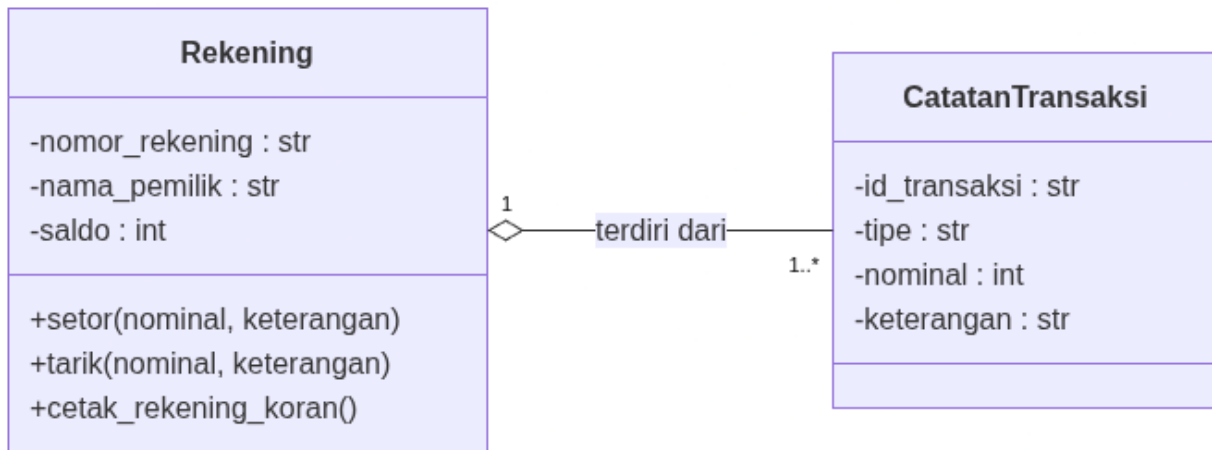
Komposisi adalah bentuk hubungan paling erat. Objek bagian tidak dapat berdiri sendiri tanpa objek induknya. Objek bagian dibuat langsung di dalam objek induk dan ikut musnah jika objek induk dihancurkan.

Ciri-ciri Komposisi:

- Objek bagian dibuat langsung di dalam objek induk
- Siklus hidup objek bagian bergantung penuh pada induknya
- Objek bagian tidak dibagi atau dipakai bersama objek induk lain



Diagram UML Komposisi:



Notasi: *-- (wajib solid di sisi induk untuk komposisi)

Contoh Implementasi Python:

```
class CatatanTransaksi:
    """Objek ini tidak memiliki arti mandiri tanpa rekening tempat mutasi
    terjadi."""

    def __init__(self, id_transaksi, tipe, nominal, keterangan):
        self.id_transaksi = id_transaksi
        self.tipe = tipe
        self.nominal = nominal
        self.keterangan = keterangan

    def __str__(self):
        simbol = "+" if self.tipe == "KREDIT" else "-"
        return f"[{self.id_transaksi}] {self.tipe:<6} {simbol}Rp{self.nominal:,} | Ket: {self.keterangan}"

class Rekening:
    """
    Rekening terdiri dari CatatanTransaksi.
    Catatan dibuat di dalam rekening dan musnah bersama rekening tersebut.
    """
```



```
def __init__(self, nomor_rekening, nama_pemilik, saldo_awal=0):
    self.nomor_rekening = nomor_rekening
    self.nama_pemilik = nama_pemilik
    self.saldo = saldo_awal
    self._riwayat = []

    if saldo_awal > 0:
        self._buat_catatan("KREDIT", saldo_awal, "Setoran awal
pembukaan rekening")

def _buat_catatan(self, tipe, nominal, keterangan):
    """Objek bagian dibuat langsung di dalam induk."""
    id_baru = f"TRX-{len(self._riwayat) + 1:04d}"
    catatan = CatatanTransaksi(id_baru, tipe, nominal, keterangan)
    self._riwayat.append(catatan)

def setor(self, nominal, keterangan="Setor tunai"):
    self.saldo += nominal
    self._buat_catatan("KREDIT", nominal, keterangan)
    print(f" Setoran Rp{nominal:,} berhasil dicatat.")

def tarik(self, nominal, keterangan="Tarik tunai"):
    if nominal <= self.saldo:
        self.saldo -= nominal
        self._buat_catatan("DEBET", nominal, keterangan)
        print(f" Penarikan Rp{nominal:,} berhasil dicatat.")
    else:
        print(" Saldo tidak mencukupi untuk transaksi.")

def cetak_rekening_koran(self):
    print(f"\n Rekening Koran: {self.nomor_rekening} an
{self.nama_pemilik}")
    print(f" Saldo Akhir: Rp{self.saldo:,}")
    print(" Riwayat Mutasi Transaksi:")
    for trx in self._riwayat:
        print(f" {trx}")

# Pembuatan objek induk secara otomatis merakit bagian-bagian internalnya
tabungan = Rekening("554-900-112", "Rina Marlina", 2000000)
tabungan.setor(500000, "Transfer masuk dari klien")
```

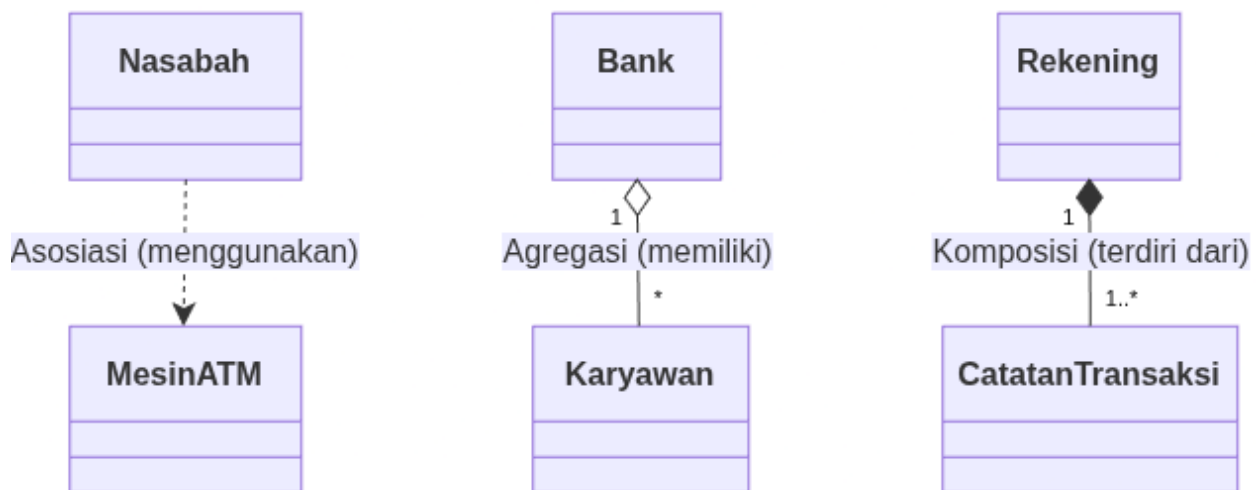


```
tabungan.tarik(300000, "Pembayaran listrik")

tabungan.cetak_rekening_koran()

# Bukti komposisi: Ketika rekening ditutup dan dihapus
del tabungan
# Riwayat CatatanTransaksi ikut terhapus karena berada di dalam instance
rekening tersebut.
```

Perbandingan Lengkap



Aspek	Asosiasi	Agregasi	Komposisi
Kata Kunci	"menggunakan"	"memiliki"	"terdiri dari"
Lokasi Pembuatan Objek Bagian	Luar objek, dikirim via parameter	Luar objek, dikirim ke penampung	Dibuat langsung di dalam objek induk
Bentuk Kepemilikan	Tidak ada	Bersama atau referensi bebas	Eksklusif milik satu induk



Dapat Hidup Mandiri	Ya	Ya	Tidak
Status Saat Objek Induk Dihapus	Objek yang dipakai tetap ada	Anggota tetap ada di memori	Objek bagian ikut musnah

Panduan Singkat Diagram Kelas UML

Notasi Atribut dan Method:

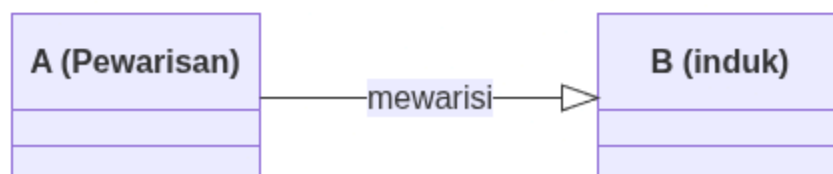
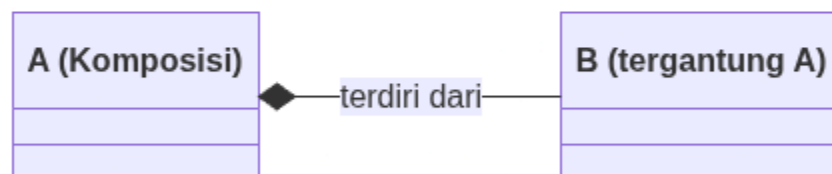
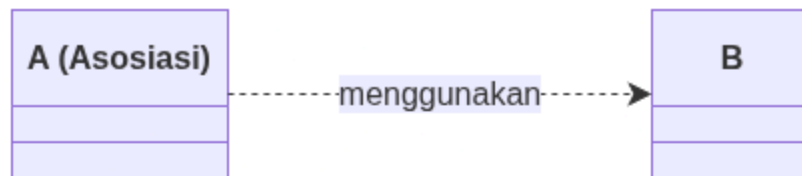
Simbol	Arti	Contoh
-	private	- nama : str
#	protected	# _saldo : float
+	public	+ tampilkan()
~	package/internal	~ _helper()

Multiplisitas (angka di garis):

Notasi	Arti
1	Tepat satu
*	Nol atau lebih (banyak)
0..1	Opsional (nol atau satu)
1..*	Satu atau lebih



Notasi





Contoh Diagram

